MECC: Expert Cache Coordination for MoE Inference on GPU

***Abstract*—HDD offloading enables mixture-of-experts (MoE) deployment on GPU beyond the task's GPU and host memory capacity. Weight reads add latency to startup and token generation. We propose MECC (MoE Expert Cache Coordination), a three-tier cache for single-GPU inference. MECC uses offline routing analysis to allocate HBM/CPU capacity across layers and initialize expert residency. CPU-to-GPU promotion reuses the freed host slot to retain the displaced GPU expert's host copy. Request-driven updates adapt residency as task demand changes. Routing and expert weights remain unchanged. On A100, MECC achieves up to 1.25× TPOT and 1.19× TTFT speedup over the state-of-the-art offloading baseline.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models increase parameter capacity through sparse activation [1]–[4]. Each token executes a few experts in each layer, but deployment requires access to the entire expert pool. The stored weights grow with the pool even when the active set stays small. CPU offloading adds host memory to GPU high-bandwidth memory (HBM), extending the space available for weights. Expert weights share memory with dense parameters, activations, and the KV cache. Once expert weights exceed this combined capacity, deployment requires a backing store [5]. Offline workloads for LLMs include model evaluation, information extraction, and form processing [5], [6]. On workstations and shared servers, a hard disk drive (HDD) can provide this backing storage. HBM and CPU memory cache part of the pool, while HDD retains its complete weights. An HBM resident executes directly. A CPU resident requires a host-to-device (H2D) copy. An expert absent from both tiers requires an HDD read followed by H2D. These paths recur as generation revisits the layers for each output token. The task manages this hierarchy within its assigned GPU and host memory. Placement determines which requests use HDD. Retention determines which fetched experts remain available for reuse.

Offloading combines expert caching [7]–[10] with CPU/GPU execution [6], [11], [12]. Recent designs use access analysis for initialization and online replacement [13], retain host copies [14], and account for storage access in replacement [15]. HDD-backed execution brings these decisions together over the lifetime of a request stream. Promotion consumes GPU space and releases host space; coordinating these events preserves reusable experts. The resulting resident set covers requests across both tiers and persists between inputs, connecting startup decisions to later weight reuse.

Three challenges arise along this lifecycle (Figure 1(a)). *Cold start* requires fetching the first input's experts from HDD when caches begin empty. *Repeated reads* occur when independently managed tiers retain overlapping entries and lose other reusable experts. After t_1, b occupies both tiers, while a has left both. Its next use at t_2 reads HDD again. *Dynamic cache adaptation* changes residency as task demand shifts. At t_3, task B requests e, b, e. Admitting e displaces b from CPU before its next use. Serving the new task then fetches both the newly selected e and the previously loaded b.

We present MECC (MoE Expert Cache Coordination), which combines a planned initial placement with request-driven caching. Figure 1(b) follows the same request lifecycle. Offline analysis counts expert selections separately for each layer. The expert planner ranks experts within each layer and allocates HBM/CPU slots by marginal demand coverage. It compares the next uncovered rank across layers to choose where an added slot covers more demand. The plan

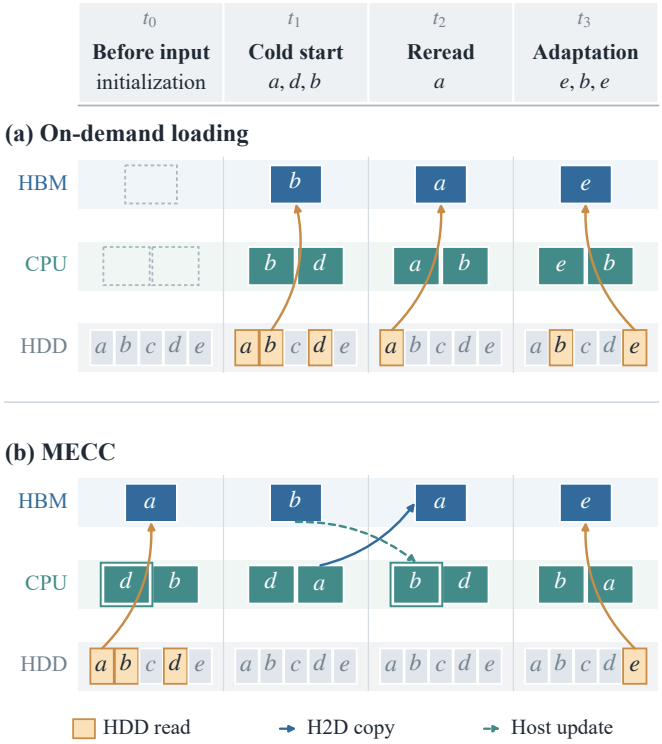


Fig. 1. On-demand loading (a) and MECC (b) over one request lifecycle. Each has one HBM slot and two CPU slots. Orange paths show HDD fetches during each stage; dashed links update host membership.

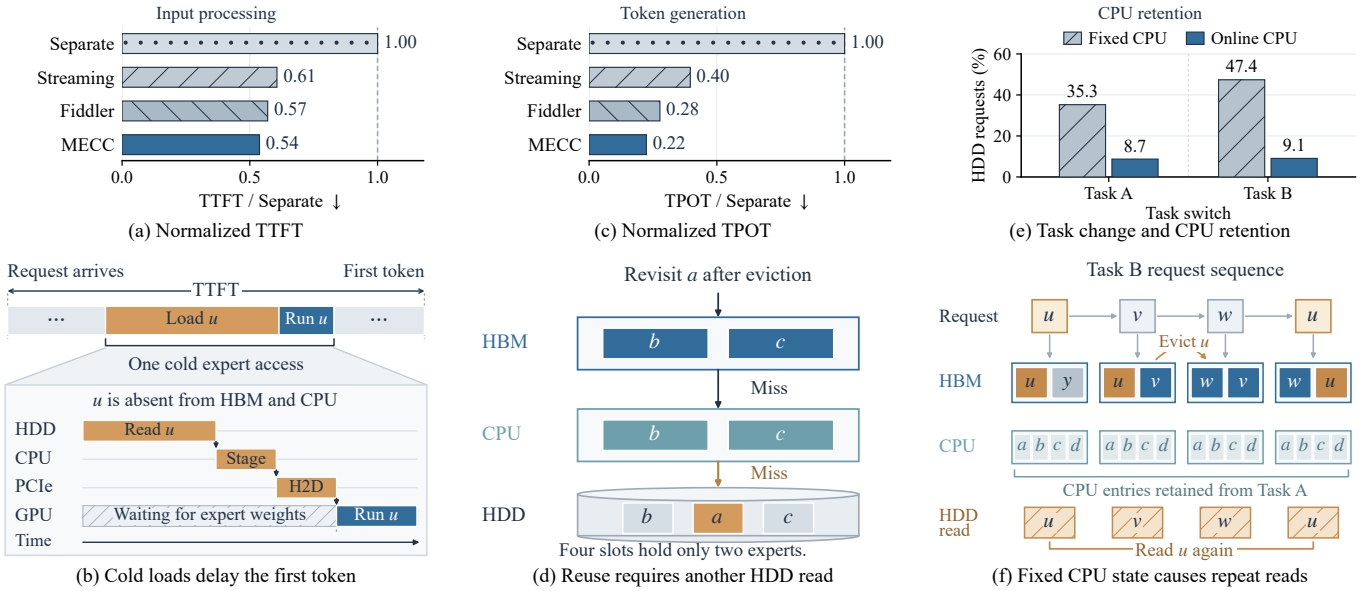
specifies per-layer capacities and the initial expert IDs. Preloading a into HBM and d, b into CPU memory covers the first input. When a promotes at t_2, the freed CPU slot retains b through a metadata update to its host mirror. Task B first loads e from HDD. Subsequent requests to b, e hit CPU after promotion and demotion update the joint recency order. This replaces the initial membership with experts selected by the new task. Per-layer capacities stay fixed, and cache state persists across tokens and requests. Weight delivery completes each required read and H2D copy before the corresponding expert executes. The router selects the experts, and each executes with its own FP16 weights. The implementation is available at https://anonymous.4open.science/r/MECC-949C/.

Our contributions are as follows.

(1) We propose an expert planner for MoE deployment on one GPU. It allocates HBM/CPU capacity across layers by marginal demand coverage and selects initial expert residents using offline routing analysis.

(2) We propose a retention mechanism that reuses CPU slots released by promotion. It links HBM replacement to CPU admission, keeping the displaced expert's host copy available for reuse and saving a device-to-host transfer.

(3) We propose a request-driven policy that adapts expert residency to changes in task demand. The policy updates the initial resident set through a joint HBM/CPU recency order, preserving per-layer capacities and cache state across tokens and requests.



(a) Normalized TTFT (c) Normalized TPOT (e) Task change and CPU retention

(b) Cold loads delay the first token (d) Reuse requires another HDD read (f) Fixed CPU state causes repeat reads

Fig. 2. Latency comparisons (a,c), CPU retention (e), and analyses (b,d,f) for HDD-backed deployment. (a,c) TTFT and TPOT relative to Separate (1.0); lower is better. (b) Cold expert access; (d) duplicate residents. (e) HDD-request shares before and after a task change. (f) Cache states during a new task.

II. Background

A. Mixture-of-Experts Inference

An MoE layer contains a router and a set of expert networks [1], [2], [16]. The router selects a subset of experts for each token and assigns a weight to each selected expert. For input x, let $G(x)$ denote the selected set and $g_e(x)$ the normalized gate weight for expert e. The layer computes

$$y(x) = \sum_{e \in G(x)} g_e(x) E_e(x). \tag{1}$$

Here E_e denotes the computation of expert e. Each expert belongs to one layer; equal indices at different layers identify different weights. Mixtral and the DeepSeek family activate a subset of their experts while retaining access to the full pool [3], [4], [17]–[19].

Autoregressive inference consists of prefill and token generation. Prefill processes the prompt and builds its KV cache. Prompt tokens assigned to the same expert share its fetched weights. Token generation processes one new token per sequence at each step and updates the KV cache. Both phases traverse the model's layers and invoke their selected experts.

B. Expert Offloading

Expert offloading stores part of the expert pool outside GPU memory and delivers selected weights to the GPU for execution. Expert weights share HBM with dense parameters, activations, and the KV cache [20], [21]. In an HDD-backed deployment, HDD retains the complete pool, while HBM and CPU memory cache resident experts. An HBM hit executes with resident weights. A CPU hit requires an H2D copy. A miss in both memory tiers requires an HDD read followed by H2D.

Initial placement selects the experts loaded before serving. Replacement updates the resident set as requests arrive. Offline selection counts determine expert ranking; runtime accesses determine recency. An expert's cache identity includes its layer and expert index. Offloading systems coordinate CPU memory, GPU execution, and weight transfers [5], [22]–[26]. Precision selection [27], [28], expert segmentation [29], [30], and staging or compression [31], [32] change the volume of transferred data.

III. Motivation

Figure 2 examines cold start, repeated reads, and changes in the active expert set.

A. Cold Start

Cold start concerns expert availability when the first input arrives. Initial residents determine which selected experts require HDD reads. Expert demand varies within and across layers [33]. A fixed slot count can cover concentrated demand in one layer and leave more accesses uncovered in another. Figure 2(a) compares Separate, Streaming, Fiddler [11], and MECC using TTFT normalized to Separate. Figure 2(b) expands one cold expert access. HDD reading, host staging, and H2D precede expert execution. The first token waits for these transfers. Startup placement therefore involves both per-layer capacities and the expert IDs assigned to those slots.

B. Repeated Reads

An expert evicted from HBM may be requested again during generation. A CPU copy serves that request through H2D. Figure 2(c) compares TPOT normalized to Separate across the same four methods. Independent replacement can retain the same experts in both tiers. In Figure 2(d), four slots hold only b and c. A later request for a misses both caches and reloads its weights from HDD. Repeated transfers add weight-delivery time to otherwise identical expert computation. Duplicate residents occupy space that could retain other reusable experts.

C. Dynamic Cache Adaptation

Task changes alter which experts receive requests. An initial ranking reflects the inputs analyzed before deployment. Figure 2(e) compares fixed and online CPU retention. Both use the same initial residents, HBM LRU, and slot budgets. Figure 2(f) follows requests u, v, w, u. The request for w replaces u in HBM. CPU memory still retains the earlier task's a, b, c, d. The final u therefore misses both caches and reads its weights again. The four CPU slots could retain all three task-B experts. Their fixed membership leaves this capacity unavailable to the new working set. Membership updates redirect these slots toward the serving task's experts.

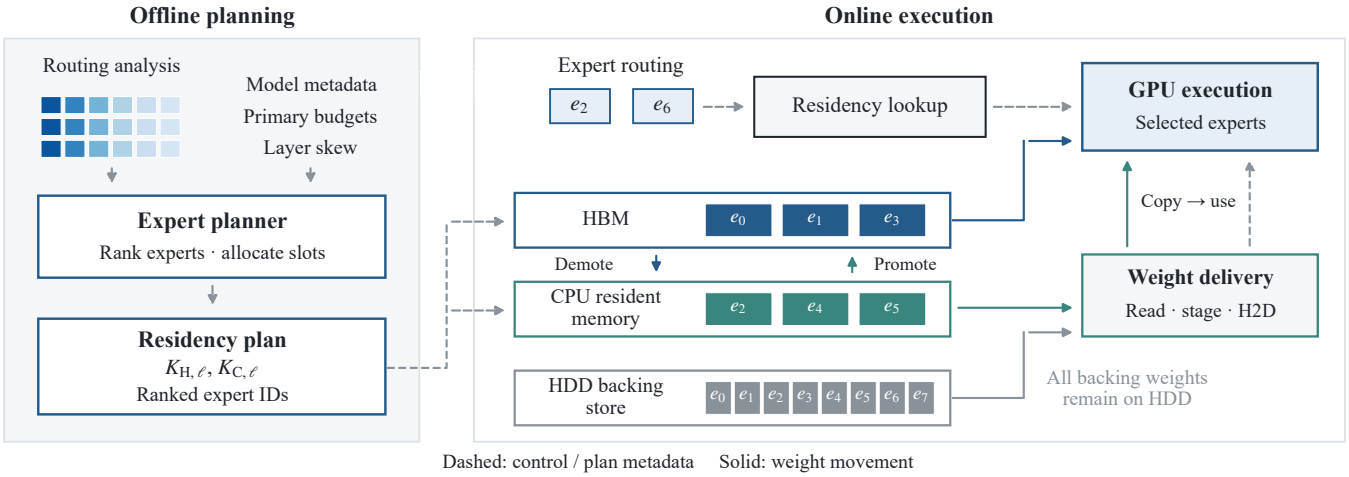


Fig. 3. MECC framework. The expert planner emits per-layer capacities and ranked IDs. Online execution maps routed expert requests to HBM, CPU resident memory, or HDD, then supplies their weights to GPU computation. Dashed arrows carry control or plan metadata; solid arrows move weights.

IV. Design

Figure 3 separates offline plan construction from online expert delivery. The expert planner receives layer/expert counts, skew parameters, model metadata, and primary residency budgets. It allocates capacities and ranks expert IDs within each layer. The loader initializes HBM and CPU from this plan. HDD retains the full expert pool and supplies weights absent from both resident tiers.

Consider the selected experts e_2 and e_6 in the framework. The lookup finds e_2 in CPU memory and e_6 on HDD. Promoting e_2 creates an HBM resident and frees its CPU slot. The runtime uses that slot to retain the expert displaced from HBM. Reading e_6 then supplies another requested expert through the HDD–CPU–GPU path. Each expert executes after its transfer completes. The model combines their outputs with the gate weights and continues to the next layer.

The cache state persists across tokens and requests. Each routed access updates expert recency within the layer's allocated capacities. An expert request carries both a layer index and a local expert ID. The same local ID in another layer refers to a different weight block. Advancing through the model preserves the earlier layers' residents. Offline planning determines the starting state; routed accesses determine its subsequent membership.

A. Expert Planner

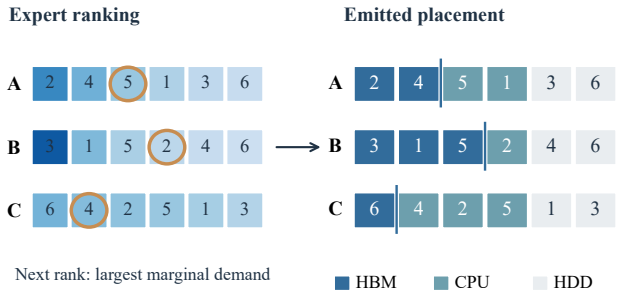


Fig. 4. Planning the initial state. The left grid contains ranked per-layer demand; circles mark the next uncovered candidates. Shared HBM/CPU slot budgets produce different cuts in the emitted rows. A, B, and C identify different layers.

Consider L MoE layers with N_ℓ experts per layer. For the routed pools considered here, experts have equal size s and layers use the same top-k.

Primary expert-residency budgets B_H and B_C provide $S_H = \lfloor B_H/s \rfloor$ HBM slots and $S_C = \lfloor B_C/s \rfloor$ CPU slots. Physical host usage includes CPU residents, GPU-weight mirrors, and staging buffers.

Calibration orders experts by selection counts. The analysis collects selected expert IDs separately for each layer. Let $\pi_\ell(r)$ denote the local ID at rank r in layer ℓ. Expert ID breaks equal-count ties. Zero-count IDs follow observed experts in ID order. The planner therefore compares demand across layers while preserving a separate expert list for each layer. A log-count versus log-rank regression supplies α_ℓ as the negative slope over positive counts. The rank model smooths the observed selection frequencies for allocation scoring:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{\sum_{j=1}^{N_\ell} j^{-\alpha_\ell}}. \tag{2}$$

Here $q_{\ell,r}$ is conditional selection mass within layer ℓ, with unit sum. Equal top-k makes this mass comparable across layers. Counts determine which expert occupies each rank; the rank model supplies the allocation scores.

Figure 4 shows the HBM and CPU cuts in those lists. Capacities satisfy $\sum_\ell K_{H,\ell} \le S_H$, $\sum_\ell K_{C,\ell} \le S_C$, and $K_{H,\ell} + K_{C,\ell} \le N_\ell$. The adaptive scan reserves one slot per layer in each fast tier, then distributes the remaining slots by marginal coverage. The reservation accounts for L slots in each tier before the remaining allowance is distributed.

The scan first assigns HBM slots. At each step, it chooses the layer with the largest next-rank score:

$$\Delta_\ell = q_{\ell,K_{H,\ell}+1}. \tag{3}$$

CPU allocation continues from the first ranks beyond each HBM cut. Its next candidate has score

$$\Delta_\ell^C = q_{\ell,K_{H,\ell}+K_{C,\ell}+1}. \tag{4}$$

The HBM offset directs CPU slots to experts beyond those already selected for the GPU. An eligible layer has an uncovered expert within its local list. The scan compares these candidates, assigns one slot, and advances the selected layer's cut. Every allocation exposes a new next-rank candidate in that layer. Layer ID breaks score ties, and expert ID breaks count ties. The direct scan takes $O(L(S_H + S_C))$ time after sorting. Its score is the additional within-layer selection mass covered by the next slot. Sorting the layer-local lists takes $O(\sum_\ell N_\ell \log N_\ell)$ time.

The expert planner serializes a residency plan: model dimensions, expert size, capacity arrays, and ranked layer-local IDs. For layer ℓ, ranks 1 through $K_{\mathrm{H},\ell}$ enter HBM. Ranks $K_{\mathrm{H},\ell}+1$ through $K_{\mathrm{H},\ell}+K_{\mathrm{C},\ell}$ enter the CPU resident set. All other ranks remain available from the backing store. The JSON plan is the deployment artifact consumed by the loader. The loader constructs a cache for each layer from the corresponding IDs and capacities. The capacity arrays remain fixed during serving. Changing task demand updates the residents through online admission.

B. Expert Retention

Each layer maintains two ordered sets, from most-recently used (MRU) to least-recently used (LRU). Their concatenation places HBM residents before CPU residents. The two primary sets contain distinct expert IDs. The ordering describes recency across both managed tiers. An HBM hit moves its expert to HBM MRU. A CPU hit removes its host entry and promotes it to HBM MRU. The evicted GPU LRU expert then becomes CPU MRU. HDD retains the backing weights for every expert. The cold payload comes from a completed read or a pending read resolved at consumption. It enters HBM MRU and follows the same demotion rule as a CPU promotion. If this admission exceeds the CPU capacity, the CPU LRU entry is removed. The backing store supplies that expert again when a later request misses both tiers.

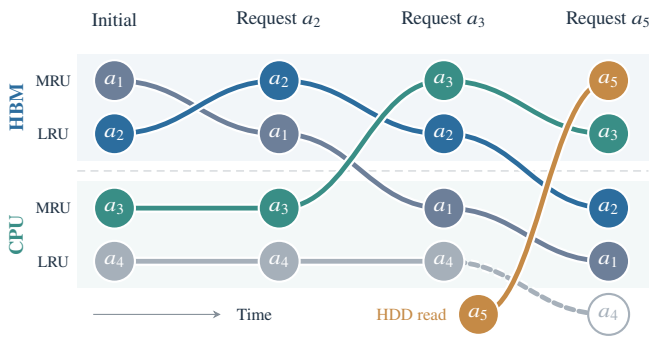


Fig. 5. Expert recency over three requests. Colored tracks follow the same experts from left to right. Within each tier, the upper position is MRU and the lower position is LRU. A CPU hit promotes a_3 and retains a_1 on the host. Loading a_5 from HDD retains a_2 and removes a_4 from CPU.

Figure 5 starts with HBM $[a_1, a_2]$ and CPU $[a_3, a_4]$. Request a_2 gives HBM $[a_2, a_1]$. Request a_3 then promotes a_3 and demotes a_1, producing HBM $[a_3, a_2]$ and CPU $[a_1, a_4]$. Finally, a_5 enters HBM, a_2 becomes CPU MRU, and a_4 leaves CPU LRU. The final lists are $[a_5, a_3]$ and $[a_2, a_1]$.

Each GPU resident carries a host mirror. The implementation stores device weights and their host tensor together in the GPU entry. On promotion, the CPU resident's host tensor continues as the promoted expert's mirror. The released slot is a logical CPU residency entry. The demoted expert's existing mirror becomes the new resident named by that entry. This exchange retains the demoted weights through a metadata update and saves a device-to-host copy.

Mirrors and CPU residents serve different roles. A mirror supports an expert currently assigned to HBM; a CPU resident supplies an expert outside the HBM set. For equal-size blocks and full resident sets, their steady host payload is $s\sum_{\ell}(K_{\mathrm{H},\ell}+K_{\mathrm{C},\ell})$. Pending reads and staging buffers add to this payload. Deployment sizing includes these allocations alongside the primary CPU capacity.

The ordered maps support lookup and recency updates in expected constant time. A CPU hit consumes one CPU entry before admitting the requested expert to HBM. A cold access introduces a new resident and may remove the CPU tail. After each access, the primary sets respect their assigned capacities. Active tensor references extend the lifetime of weights through expert execution.

C. Online Execution

Serving requests update the initial state through the same access rules. When a task selects different experts, its first accesses admit them to HBM. Subsequent demotions place them at CPU MRU, while older CPU entries move toward replacement. HBM hits refresh recency directly. The updated state survives the end of a prompt and serves later requests on the same model instance.

For each MoE layer, the wrapper evaluates the gate and groups token rows by selected expert. The loader resolves each layer-local expert ID through the memory tiers. The expert kernel processes its assigned rows. Gate weights scale the outputs before accumulation into the layer result. The gate continues to determine expert selection and mixture weights.

During prefill, the loader submits reads for the selected experts absent from memory. Each read is resolved when its expert reaches consumption. Figure 6 shows resident work overlapping a pending read for another selected expert.

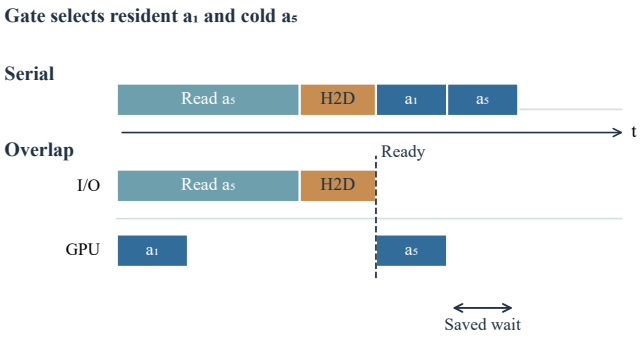


Fig. 6. Prefill delivery for resident a_1 and cold a_5. Computing a_1 overlaps the pending read for a_5. H2D completion releases a_5 for GPU execution; the shortened schedule saves exposed waiting time.

A read future returns the host tensors that supply H2D. The copy stream moves them to the GPU. The loader synchronizes pending copies before yielding device tensors to computation. The expert kernel then consumes those tensors. Recording the consumer stream keeps their storage live through kernel completion. These dependencies connect read completion, copy completion, and expert execution.

Prefill can request more experts than a single resident set holds. The loader batches CPU-resident experts when the group fits the configured HBM slot count. Larger groups use sequential access and eviction. Temporary transfer buffers and active references form part of runtime memory use alongside managed residents. Token generation uses the selected experts available for the current step. An HBM hit executes with existing device weights; other accesses take the corresponding host or HDD delivery path.

Prefetchers use access analysis or lookahead to anticipate expert demand [34]–[36]. Batch and request schedulers expose work from concurrent inputs [37], [38]; distributed edge schemes add network delivery [39]. MECC combines request-driven retention with delivery of the experts selected at the current gate. Retention determines which weight blocks remain available across accesses. Delivery schedules the reads and copies that those accesses require. Recency updates change metadata and resident references. Reads and copies provide a new physical placement when the selected expert requires one.

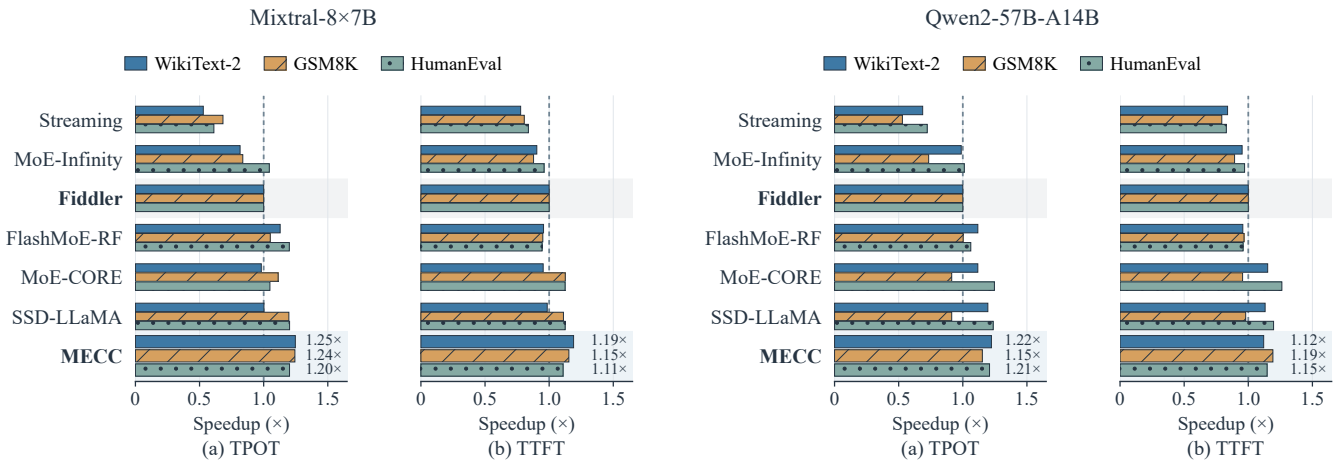


Fig. 7. A100 end-to-end comparison. The left pair shows Mixtral-8×7B; the right pair shows Qwen2-57B-A14B. Each bar is speedup over Fiddler for the matched model, workload, and metric; higher is better.

V. Evaluation

A. Experimental Setup

Hardware. The A100 experiments use one NVIDIA A100 80 GB PCIe GPU and an HDD RAID backing store. The large-model experiment uses one NVIDIA H100 SXM 80 GB GPU with 3.35 TB/s HBM3 bandwidth and a PCIe Gen5 host link. Its HDD RAID sustains about 425 MiB/s sequential reads. Expert weights remain in FP16 throughout.

Models and workloads. A100 runs cover Mixtral-8×7B and Qwen2-57B-A14B on WikiText-2, GSM8K, and HumanEval. The H100 run uses the 141B-parameter Mixtral-8×22B-Instruct-v0.1 with 56 layers, eight experts per layer, and top-2 routing. Each expert occupies 576 MiB, yielding about 252 GiB of FP16 expert weights. The primary HBM and host budgets are 50 and 64 GiB; the allocation gives each layer one HBM slot and two host slots, leaving five experts per layer on HDD.

Baselines. The A100 comparison includes on-demand Streaming, Fiddler, MoE-CORE, MoE-Infinity, SSD-LLaMA, and FlashMoE-RF [9], [11], [13]–[15]. The H100 comparison includes Streaming, Fiddler, FlashMoE-RF, and MECC. Streaming loads each requested expert from the backing store when it is needed.

Metrics. TTFT measures time to the first token; TPOT averages the time per subsequent token. Both include computation and expert delivery, and we average latency across prompts. Figure 7 reports $T_{\text{Fiddler}}/T_{\text{method}}$ for each model, workload, and metric; values above one indicate lower latency than Fiddler. Figure 8 uses the same ratio for H100.

B. End-to-End Performance

A100 model–workload comparison. On Mixtral-8×7B, MECC provides 1.20–1.25× TPOT and 1.11–1.19× TTFT speedup over Fiddler across the three workloads (Figure 7, left). The corresponding Qwen2-57B-A14B ranges are 1.15–1.22× and 1.12–1.19× (Figure 7, right). Mixtral's TPOT ratios cluster within 0.05×, whereas its TTFT ratio moves from 1.19× on WikiText-2 to 1.11× on HumanEval. Qwen2 responds differently: GSM8K gives MECC its smallest TPOT ratio and largest TTFT ratio among the three tasks. These differences locate the benefit in the response stage: prompt processing and autoregressive decoding expose different expert-request sequences to the same residency policy.

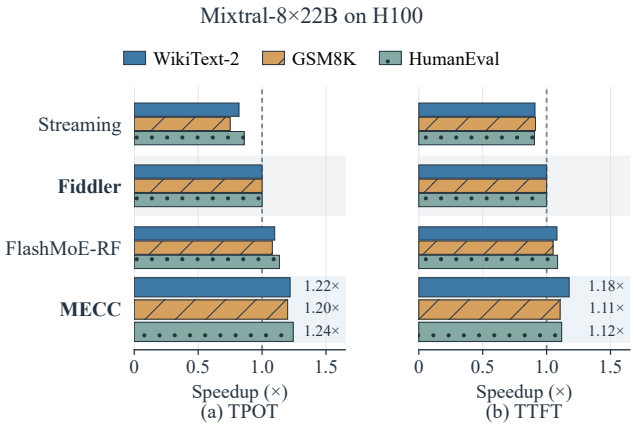


Fig. 8. H100 SXM 80 GB comparison on Mixtral-8×22B-Instruct-v0.1 across WikiText-2, GSM8K, and HumanEval at 128/128 tokens. Colors and hatches follow Figure 7; each bar is $T_{\text{Fiddler}}/T_{\text{method}}$. Higher is better.

H100 large-model comparison. The approximately 252 GiB expert pool exceeds the 50 GiB HBM and 64 GiB CPU primary pools combined, so the backing store remains necessary. For WikiText-2, GSM8K, and HumanEval, MECC's TPOT is 82, 90, and 74 s/token versus Fiddler's 100, 108, and 92 s/token: 1.22×, 1.20×, and 1.24× speedup, respectively (Figure 8). MECC's TTFT is 17, 19, and 17 min versus Fiddler's 20, 21, and 19 min, corresponding to 1.18×, 1.11×, and 1.12×. The workload order differs between TPOT and TTFT, consistent with repeated expert demand during decoding and cold accesses before the first token placing different pressure on the resident tiers.

Comparison with FlashMoE-RF. On H100, FlashMoE-RF records 91, 100, and 81 s/token TPOT and 18.5, 20, and 17.5 min TTFT in the same workload order. MECC reduces these values by 9, 10, and 7 s/token and by 1.5, 1.0, and 0.5 min, respectively. The first-token margin narrows from WikiText-2 to HumanEval, while the decode reduction remains at least 7 s/token.

Capacity and stage effects. Only three of eight experts per layer can reside in HBM or host memory at once, so a change in the routed pair can require an HDD fetch even after the initial placement. TTFT includes the cost of establishing the first working set; TPOT also reflects whether later requests reuse experts retained in the faster tiers.

C. Ablation Study

We add placement, joint retention, and delivery scheduling in sequence, then measure request sources and exposed time under the same configurations.

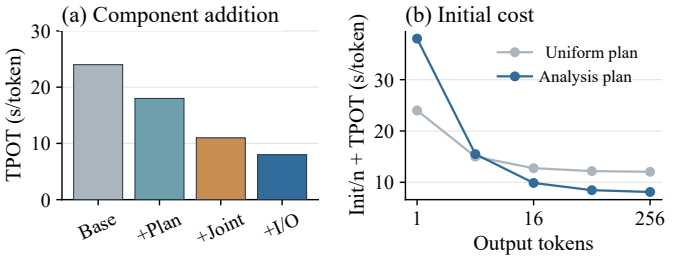


Fig. 9. Component addition and initialization amortization. (a) Adding placement, joint retention, and delivery scheduling. (b) Initialization plus generation over output length.

Component addition. Figure 9(a) shows the TPOT change after each component is added. The largest reduction follows joint retention, which keeps a displaced GPU expert in the host tier. The plan changes the starting residents; joint retention then changes which experts remain accessible after a GPU replacement. This distinction matters because a useful initial set alone cannot preserve an expert once subsequent requests displace it. Panel (b) includes initialization cost: over n output tokens, its amortized contribution is T_{init}/n per token.

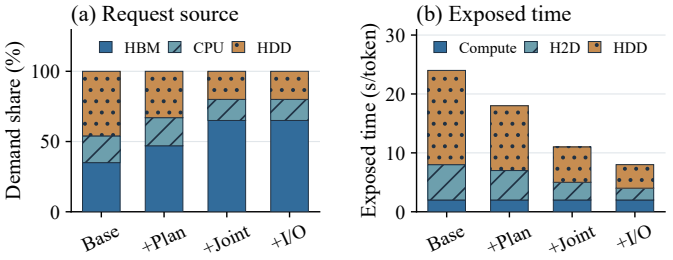
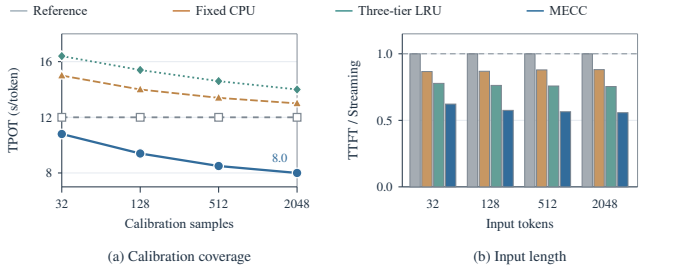


Fig. 10. Expert request sources and exposed time across the same component sequence.

Traffic and delivery. Figure 10(a) shows the fraction of expert requests served from HBM, CPU, and HDD after each component is added. Panel (b) breaks exposed time into computation, host-to-device transfer, and HDD access. The final delivery step changes exposed time without changing the request-source fractions, separating scheduling effects from changes in cache residency. Thus, a lower HDD demand share and a shorter exposed HDD interval are distinct effects: the former changes where requests hit, while the latter changes how much of a required read remains on the critical path.

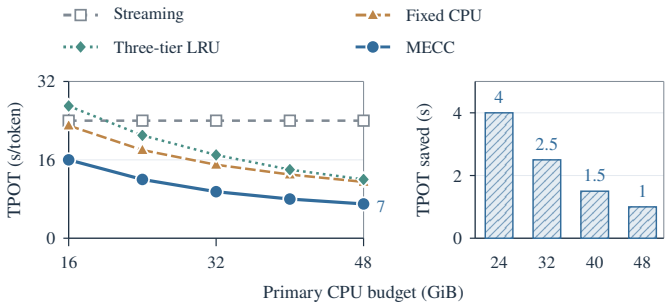
D. Sensitivity Analysis



(a) Calibration coverage (b) Input length

Fig. 11. Sensitivity to analysis samples and input length. TTFT uses the Streaming reference at each input length.

Analysis and request shape. Figure 11(a) increases the number of analysis samples from 32 to 2048; TPOT decreases from 10.8 to 8.0 s/token. Panel (b) reports TTFT relative to Streaming at each input length, so each point uses a length-matched denominator. The TPOT curve flattens as the analysis set grows, consistent with later samples making smaller changes to the selected initial residents.



(a) Decode latency (b) MECC savings

Fig. 12. CPU capacity sensitivity. (a) TPOT at a fixed HBM budget. (b) Savings from each additional allocation.

Host capacity. Figure 12 increases primary CPU residency from 16 to 48 GiB at a fixed HBM budget; MECC TPOT falls from 16 to 7 s/token. The first 8 GiB increment saves 4 s/token, whereas the last saves 1 s/token. The diminishing reduction is consistent with additional slots covering less frequently reused experts.

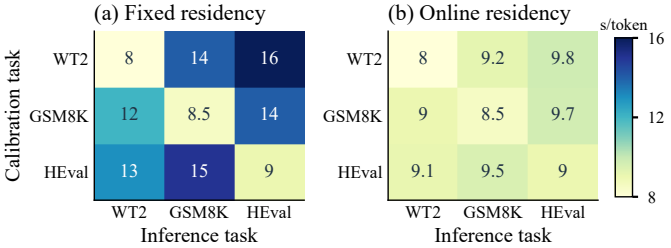


Fig. 13. Task transfer. Rows select the analysis task; columns select the inference task. Both panels share a TPOT scale.

Task transfer. In Figure 13, rows specify the analysis task and columns the inference task. Across the six off-diagonal pairs, online membership reduces mean TPOT from 14.00 to 9.38 s/token, a 33.0% decrease; the diagonal entries are unchanged. For the HumanEval inference column, the largest fixed-residency value falls from 16 to 9.8 s/token with online updates. An access to an expert outside the initial set admits it to the resident hierarchy; later requests can then reuse the new placement. The unchanged diagonal cells and reduced off-diagonal values separate adaptation to a changed task from simply increasing the slot budget.

VI. Conclusion

MECC couples CPU promotion with retention of the displaced GPU expert in the freed host slot. A host mirror supports this update without a device-to-host copy. Placement planning supplies the initial residents; serving requests update their membership. Across the models and workloads in Figure 7, MECC improves both TPOT and TTFT relative to the matched Fiddler reference. On H100 with Mixtral-8×22B, MECC improves TPOT by 1.20–1.24× and TTFT by 1.11–1.18× over Fiddler across the three workloads. Component, capacity, and task-transfer studies characterize placement, cross-tier retention, delivery, and adaptation.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[6] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[7] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[8] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[9] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[10] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[11] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[12] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[13] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[14] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[15] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[16] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[17] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[18] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[19] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[20] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[21] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[22] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[23] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[24] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[25] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[26] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.

[27] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[28] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[29] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[30] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[31] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[32] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[33] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[34] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[35] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[36] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[37] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[38] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[39] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.